Reporte técnico — Nebula Arena en la nube

Unidad 3: Infraestructura y despliegue de videojuegos en la nube.

Todas las cifras de este reporte salen de mediciones o de precios oficiales, con su fecha. Lo que no se midió está marcado como **no medido**.

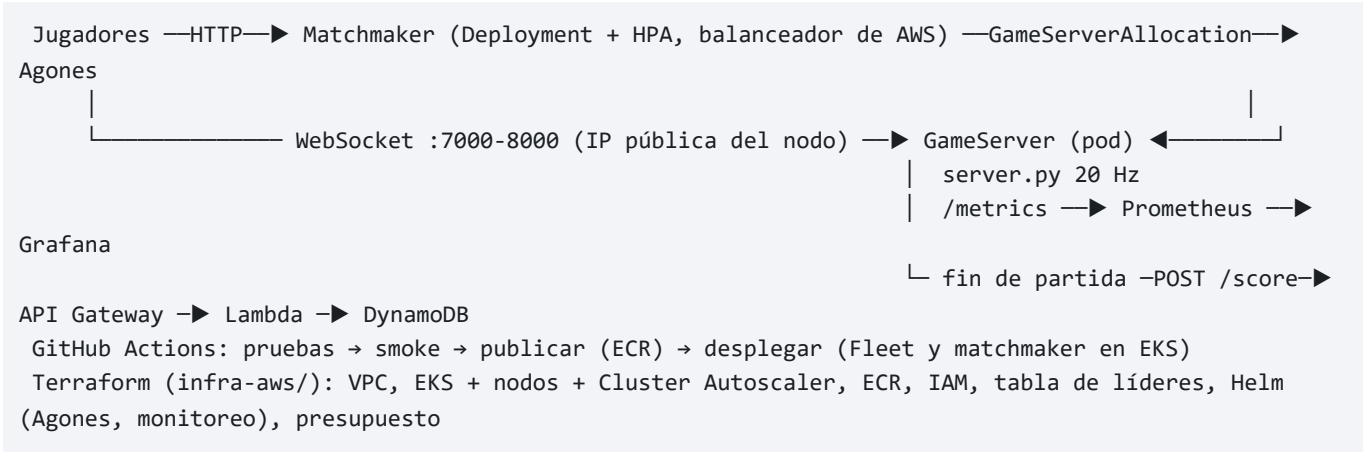
1. Resumen

Nebula Arena es un shooter de arena 5 vs 5 en 3D con bots, que sirve como caso de estudio. El servidor de partida es autoritativo (Python asyncio + WebSocket, 20 ticks/s) y corre en contenedores administrados por **Agones** sobre **Amazon EKS**.

El resto de la plataforma: - **Escalado:** en tres capas (flota, pods y nodos). - **Métricas:** Prometheus y Grafana. - **Pruebas de carga:** Locust. - **Publicación:** GitHub Actions hacia ECR. - **Tabla de líderes:** Lambda + API Gateway + DynamoDB. - **Infraestructura:** toda se crea y se destruye con **Terraform**.

Estado (2026-11-09): todo se desplegó y se probó en AWS (us-east-2, la única región que permite la cuenta del plan gratuito) y después se destruyó con `terraform destroy`. La variante para Google Cloud del material quedó escrita en `infra/` como alternativa: la cuenta de Google pidió un prepago.

2. Arquitectura



Capa	Componente	Archivo
Juego	Servidor autoritativo, 5 clases, bots de reglas con A*	`server/`
Contenedor	`python:3.11-slim`, usuario sin privilegios	`server/Dockerfile`
Orquestación	Fleet + FleetAutoscaler (buffer)	`k8s/aws/fleet.yaml`, `k8s/autoscaler.yaml`
Servicio sin estado	Matchmaker con HPA por CPU	`matchmaking/`, `k8s/aws/matchmaker.yaml`
Métricas	PodMonitor + dashboard (14 paneles)	`k8s/podmonitor.yaml`, `monitoring/`
Carga	Locust: jugadores (y espectadores, no ejecutado)	`loadtest/`
Serverless	Tabla de líderes: `POST /score`, `GET /top`	`serverless/aws/`
IaC	Terraform con estado remoto en S3	`infra-aws/`

Decisiones principales - **Servidor autoritativo:** el cliente no decide el daño ni las posiciones. Se rechazan teletransportes, paredes atravesadas y mensajes inválidos. - **Drenado:** el servidor llama a `/shutdown` solo cuando la partida termina, así Agones nunca recicla un servidor con jugadores. - **Red pública sin NAT Gateway:** ahorra ~USD 33 al mes. Los servidores de juego necesitan IP pública de todos modos. - **Credenciales:** - Terraform usa credenciales temporales (`aws login`). - La llave de la tabla de líderes vive en SSM, fuera del código y del estado. - GitHub tiene un usuario IAM con permisos mínimos. OIDC estaba escrito, pero el plan gratuito lo bloquea con una política de la organización.

3. Práctica 3.1 — Bot e inferencia

Medición (Colab, T4)	Resultado
Entrenamiento CPU vs GPU	**La GPU fue 68 % más lenta** (red pequeña: copiar a la GPU cuesta más que calcular)
Capacidad por vCPU de nube	2,315–2,787 inferencias/s
2,000 bots a 10 Hz	20,000 inferencias/s → **12 vCPU** con 30 % de margen
Costo (precios oficiales de GCP, us-central1, 2026-11-09)	3 × `c3-standard-4` = **USD 0.6048/h**, **USD 0.0003 por bot y hora**

Los bots del juego son de reglas (máquina de estados con A*). El modelo de la 3.1 solo se usó para medir el costo de la inferencia. Detalle en `docs/practica-3.1.md`.

4. Contenedores y orquestación (3.3)

EKS 1.36, nodos `m7i-flex.large` (2 vCPU, 8 GB), Agones 1.62:

- **Arranque:** la flota tuvo 3 servidores `Ready` a los 8 s del `kubectl apply`.
- **Partida real por internet desde Colima:** 10 jugadores durante 15 s, RTT p50 de 106 ms, p95 de 133 ms y p99 de 147 ms.

- **Ciclo completo:** `Ready → Allocated → Shutdown`, y la flota repuso el servidor sola.

Evidencia: `entrega/evidencias/3.3-cluster-agones/`.

5. CI/CD (3.2)

`pruebas` (107 pruebas de pytest) → `smoke` (imagen + partida de 10 jugadores contra el contenedor; falla si el tick rate baja de 19) → `publicar` (ECR, imagen con el SHA del commit) → `desplegar` (parcha la Fleet y el matchmaker en EKS y espera servidores `Ready` con la versión nueva).

- **Corrida en verde:** run 42694334223, commit `c295bf8`, **1 min 33 s** de punta a punta.
- **Imágenes inmutables:** ECR no deja reescribir una etiqueta, así que cada versión se rastrea a su commit.

Rollback: se vuelve a desplegar la imagen del commit anterior, que sigue en ECR con su SHA, o se revierte el commit para que el pipeline la vuelva a publicar. Agones reemplaza solo los servidores libres; las partidas en curso terminan con la versión que tenían.

6. Autoescalado y pruebas de carga (3.4)

Rampa en EKS: 300 jugadores simulados, 1 nuevo por segundo, 10 minutos. Los nodos tenían un máximo de 6, elegido para cuidar los créditos.

Capa	Antes	Pico	Tiempo de reacción
Flota (FleetAutoscaler)	3 servidores	37 (34 en partida)	43 s hasta el primer escalado
Pods (HPA del matchmaker)	2 réplicas	2 (CPU 2–3 % de 60 %)	no hizo falta
Nodos (Cluster Autoscaler)	4	6 (máximo)	~1 min por nodo (4→5 y 5→6 con un minuto de diferencia)

Resultado	Valor
CCU máximo	**252** jugadores al mismo tiempo
Capacidad por nodo	~6 servidores × ~7.4 jugadores ≈ **42 jugadores por nodo** m7i-flex.large (el límite son las peticiones de CPU de 250m por servidor)
Tick p99 del servidor	≤ 2.5 ms (5 % del presupuesto de 50 ms)
RTT hasta 164 jugadores	p50 120 ms, p95 320 ms
RTT con más de 200 jugadores	de segundos (ver hallazgo 3)

Hallazgos 1. **El techo fue el máximo de nodos.** A partir de ~250 jugadores los servidores nuevos se quedaron `Pending` y el matchmaker respondió 503 (198 de 234 asignaciones con 503). Con 42 jugadores por nodo, **1,000 jugadores necesitan ~24 nodos**. 2. **Los servidores nunca se saturaron.** El tick rate se mantuvo

en 20 y la CPU máxima de un nodo fue de 74 %. 3. **La latencia alta arriba de 200 jugadores viene, con mayor probabilidad, del generador de carga** (una laptop con 4 procesos de Locust por internet doméstico). Las peticiones por segundo de Locust cayeron de 55 a 7 mientras el servidor seguía en 20 Hz. **No está comprobado:** se confirmaría corriendo Locust en una VM dentro de AWS. 4. **La meta de p95 < 80 ms no es alcanzable desde Colima con us-east-2.** El RTT base es de ~106 ms sin carga. Para jugadores en México haría falta una región más cercana, que el plan gratuito no permite. 5. **EKS no trae metrics-server.** Sin él, el HPA no veía la CPU. Se agregó al Terraform.

En los ensayos locales (kind) las pruebas de carga encontraron y corrigieron dos fallas reales: - `/ready` se llamaba una sola vez, y los servidores se quedaban atorados en `Scheduled`. - Python como PID 1 ignoraba `SIGTERM`, así que cada servidor tardaba 30 s en morir.

No medido: - Rampa de 1,000 jugadores y pico de 1× a 10×. Necesitan ~24 nodos; se limitó a 6 para no agotar el crédito. - 500 espectadores por CDN.

Extrapolación de espectadores (cálculo): 20,000 espectadores a 1080p (6,000 kbps) durante 1 hora = 20,000 × 6,000 × 3,600 / 8×10^6 = **54,000 GB ≈ 54 TB por hora** (120 Gbps). Debe servirse desde una CDN (CloudFront), nunca desde los servidores de juego.

7. Tabla de líderes serverless (3.5)

API Gateway (HTTP) → Lambda (Python 3.12) → DynamoDB, con 20 pruebas en verde. Las escrituras se protegen con una llave en SSM y se usan sumas atómicas (`ADD`).

Prueba 80/20 con 500 usuarios (desde la PC):

	Antes	Después de la mejora
Atendidas con éxito	~35/s	**~99/s**
Errores 429	102,607	0
Duración de la Lambda	50 ms	13 ms
Latencia p50 / p95 / p99	75 / 130 / 260 ms	77 / 110 / 210 ms

- **Mejora aplicada:** caché del top de 2 s y límite de API Gateway subido a 1000 req/s.
- **Cuello de botella restante:** la cuota de **10 ejecuciones simultáneas** de Lambda de la cuenta nueva (lo normal es 1000; se pide en Service Quotas).
- **Arranque en frío:** hasta 3.5 s en la primera ráfaga.

8. Infraestructura como código (3.6)

`infra-aws/` crea **46 recursos** con un `terraform apply`: red, EKS, nodos, Agones, Cluster Autoscaler, monitoreo, ECR, IAM, tabla de líderes y presupuesto.

- **Estado remoto en S3** con bloqueo nativo (`use_lockfile`): bucket privado, versionado y cifrado.
- **Drift:** el `plan` contra la infraestructura viva, después de la prueba de carga, no encontró cambios hechos a mano.

- **Idempotencia:** un segundo `plan` respondió `No changes`.
- **Destroy:** `terraform destroy` borró los 46 recursos, y la revisión en AWS no encontró nada del proyecto cobrando (`04-destroy.txt`). Encontró dos fallas que se corrigieron: Terraform borraba la ruta a internet antes que Agones, y `force_destroy` del usuario IAM choca con la política del plan gratuito.

9. Costos

Precios bajo demanda de AWS en us-east-2, consultados con la API de precios el 2026-11-09:

Concepto	Valor	Fuente
Clúster en reposo (EKS + 2 nodos + IPs + balanceador)	**USD 0.34/h** (~USD 8/día)	API de precios
Clúster en el pico (6 nodos)	**USD 0.74/h**	API de precios
Costo por jugador y hora	0.74 / 252 ≈ **USD 0.003**	pico medido en la 3.4
1,000 jugadores (~24 nodos, estimado)	≈ USD 2.5/h	24 × 0.096 + EKS + IPs
Tabla de líderes con 5 M consultas/mes	≈ USD 6–7/mes serverless vs ≈ 15–20 una VM 24/7	precios de lista
Inferencia de 2,000 bots	USD 0.60/h	3.1 (GCP)
Egress de 20,000 espectadores	54 TB/h; **no cotizado** con la tarifa de CloudFront	—
Gasto real del proyecto (1–9 nov)	≈ USD 3.3 antes de créditos, sin contar EKS del día 9 (Cost Explorer tarda 24 h)	Cost Explorer

Control de gasto: - Presupuesto en Terraform que mide el costo antes de créditos y avisa al 50 %, al 80 % y al 100 % del pronóstico. - `destroy` al terminar cada sesión. - Un máximo de nodos en el autoescalador.

10. Recomendaciones

- **Tipo de máquina:** el servidor usa ~2.5 ms de los 50 ms por tick, así que la petición de 250m de CPU por servidor sobra. Bajarla a ~100m duplicaría o triplicaría los jugadores por nodo; se confirma con otra rampa.
- **Mínimo de nodos:** 2 en reposo; el máximo, según el pico esperado (~24 para 1,000 jugadores con la petición actual).
- **Buffer de la flota:** la flota pasó 93 % de la rampa sin servidores `Ready` libres. Conviene un buffer mayor o escalado programado antes de picos esperados.
- **Generar la carga desde la nube** (una VM en la misma región) para separar la latencia del generador de la del sistema.
- **Servidores de partida bajo demanda, no Spot** (una interrupción corta la partida). El entrenamiento de bots sí puede ir en Spot.
- **Tabla de líderes:** pedir la cuota de concurrencia de Lambda y cambiar el `scan` por un índice ordenado o Redis *sorted sets*.

11. Conclusiones

La plataforma completa funcionó en AWS de punta a punta: - **Despliegue:** el pipeline publica la imagen y actualiza la flota sin cortar partidas. - **Autoescalado:** las tres capas escalaron solas con 300 jugadores, y los servidores nunca se saturaron. - **Tabla de líderes:** aguanta ~99 escrituras y lecturas por segundo con la cuota mínima de una cuenta nueva. - **Infraestructura:** todo se crea y se destruye con Terraform, sin drift.

Los límites encontrados fueron de la cuenta (región, cuota de Lambda, tope de nodos elegido por costo) y del generador de carga, no del diseño. Las pruebas de carga sirvieron para lo que deben servir: encontraron fallas reales (`/ready` , `SIGTERM` , metrics-server, el `scan` del top) que se corrigieron.